

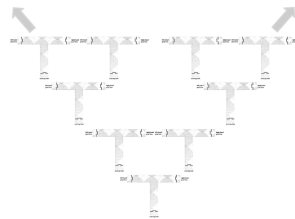
Course Notes — SESSION 3

SEARCH

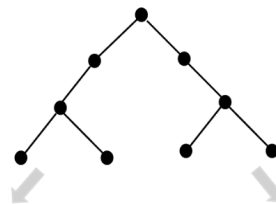
1. Introduction

The previous session introduced a view of credit assignment as processing syntactic (bits) or semantic (physical symbols) information. This view implied the reinforcement learning paradigm in AI is driven by brute force (big computers), while the learning-in-a-society paradigm is driven more by having good abstractions (languages). In this session, we turn to a further AI idea about information processing called **search**. Studying search will illuminate fundamental ways to assign credit in either paradigm more efficiently (i.e., to reduce the need for brute force, and to make better use of abstractions), as well as to surface some distinct limits.

To see what is meant by search, imagine we have stacked a large number of T-mazes, such that our mouse's problem of finding the cheese is complex to the point of being combinatorially explosive. Let us change, however, how we visualize the maze. First, each T-maze will be a simple point that we will call a “**node**”. Second, we will now flip the maze upside down, so the mouse starts from a single action on top, and can select among various “**paths**” of nodes that correspond to possible action sequences.



Combinatorially-explosive T-mazes



The same picture, as nodes of a search tree

What do you see in this new picture? One neat thing is that the maze looks something like a **hierarchical division of labor** (later, we will call this a **search tree**) — a rough version of Minsky's idea of a “society”. Think of the top node as our mouse “CEO”; the levels beneath it as middle managers; and further down, staff. The mice navigate the maze by sending and receiving messages up and down this hierarchy. They specialize in simple problems, corresponding to some level of nodes in the maze, and controlled by levels above it. This picture gives a first intuition of the AI idea of search:

If a problem is too complex, divide it into simpler ones.

Next, consider that we need not divide the labor for each mouse to just a single level. A mouse with some processing capacity can do “**planning**” — *looking ahead by evaluating possible sequences of actions* — across at least some subset of interdependent nodes. By remembering good sequences of actions, the mouse can have beliefs about its **plans** and

not just individual actions. Hence, the mice can divide the labor of the problem in many possible ways, according to the plans that they are able to make:

If a plan (sequence of actions) for solving a problem is too complex, divide it into simpler plans.

Finally, assume that, as this organization of mice learns to plan its way through this maze, it also learns beliefs about individual actions at certain nodes of the maze, and later uses these beliefs to inform how they navigate other similar mazes. The beliefs may not be perfect, but “**good enough**” for the mice to use them as a stable basis for making plans in these other mazes. In search, such imperfect but stable beliefs about problems are called **heuristics**, or rules-of-thumb for guiding individual actions. Using heuristics *economizes on information processing capacity*, in that starting closer to a solution to a problem (not just having 50-50 beliefs about whether to go left or right at each node of the maze) reduces the need for costly learning and, hence, is more efficient:

Agents make plans (sequences of actions) to solve problems efficiently (economizing on information processing) by learning and using heuristics.

The above discussion explains what will be meant by the AI idea of **search** in our course: *dividing the labor of solving a complex problem into plans, using heuristics to economize on information processing*. Our questions for this session, hence, will be: What is meant by “plans” and “heuristics”? What does a “search process” look like? And what implications does search have for the credit assignment problem?

To study these questions robustly, we will, once again, consider two paradigmatically-different approaches. A first approach, which we will call **von Neumann search**, after one of its pioneers, the mathematician John von Neumann (1903-1957), conjures search as *playing a well-defined game*, such as a player searching for a sequence of moves in chess. This approach extends our previously studied ideas of reinforcement learning, Shannon information, and of credit assignment as a matter of brute force. A second approach, which we will call **Simon search**, after Herbert Simon (1916-2001), instead conjures search as an *open-ended process of understanding and structuring an ill-defined problem*, such as an entrepreneur developing a new product for an uncertain market.

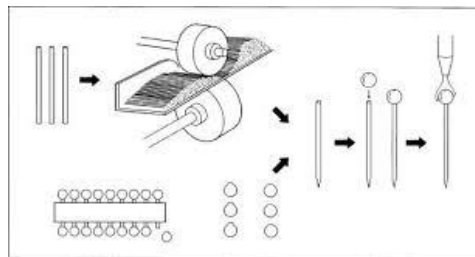
2. Plans: dividing the labor of problem-solving

In AI textbooks, search is commonly introduced by the example of planning the shortest route to travel from A to B, such as from home to the airport, where the nodes in between are various intermediate locations such as a bus stop or train station. This course will take

a different approach, focusing instead on our fundamental analogy between AI and organizations. As implied in the section above, search for us will be narrated as the making of plans based on some division of labor and, hence, as about coordinating interdependent tasks. To make this story stick, we will need a somewhat unfamiliar shift in perspective — one that takes a detour into 19th century industrial factories and early examples of computers. The payoff of this detour will be a basis for understanding the foundational AI idea of search less in terms of optimizing a route from A to B, and more as a fundamental approach to assigning credit for any complex real-world problem.

2.1 The division of labor as a general solution to credit assignment

The modern concept of dividing the labor of a problem dates back to the start of the industrial revolution in 18th century Europe. The Scottish economist Adam Smith explained the logic in his 1776 book *The Wealth of Nations*, giving the example of a stylized pin factory that, as depicted below, divides the problem of making pins into tasks involve making pin heads and pin bases, and tasks of assembling them.



Tasks of (1) making pin bases; (2) making pinheads; and (3) assembling pins in a pin factory.

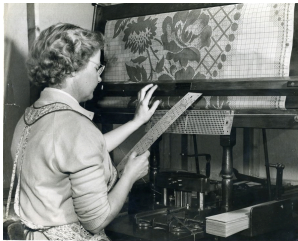
Smith considers two strategies for organizing the three tasks. In the first strategy, three “**generalists**” are used, meaning each worker makes both the head and the base, and who assembles the two parts. In the second strategy, the labor of each the three tasks is divided across three “**specialists**”, meaning each focuses on just one of the tasks. Smith explained how the latter is more economical thanks to **gains to specialization**. Why? In the parlance of our course: *simpler tasks have less interdependencies by definition, which reduces the complexity of the credit assignment problem*. Put another way, simple tasks better enable learning how to do the tasks well. Hence, it is not just that a simple task done by a specialist is easier to *perform*, but that it is easier to *learn to perform*.

Building on Smith’s insight on the gains to specialization, 19th century English economist Charles Babbage made three bold proposals in a famous 1834 book titled *On the Economy of Machinery and Manufactures*. First, Babbage proposed, **any complex task can be divided into simpler subtasks**, such that gains to specialization can be achieved for any problem of any complexity. In the parlance of our course, the division of labor is a general solution to the problem of learning to assign credit. Second, once one

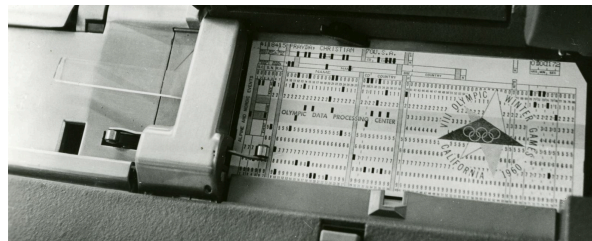
has learned to divide labor into simple enough tasks, **any task can be mechanized** (i.e., built into an AI system). Third, this logic applies not just to physical tasks like making pins, but equally to any *problems of thinking*; further, if thinking problems can be divided into simple enough *mental tasks*, it is possible to mechanize thought by building a general purpose “**thinking machine**”.

2.2 Dividing labor into plans (programs)

Babbage’s analytic engine was inspired by a newly invented technology at the time called the **Jacquard loom**, in which the pattern to be woven into a fabric could be mechanized as a sequence of instructions for actions corresponding to holes punched into a card, as depicted below. Babbage called this sequence of instructions as the **algorithm**, which you can think of as some “precise and unambiguous specification for controlling a sequence of steps that can be carried out mechanically” (XX).



A program being inserted in a Jacquard loom for making a fabric pattern.



A punched card produced by IBM to store programs, and widely used in digital computers into the 1960s.

Babbage proposed further that physically implementing **plans** (i.e., mechanizing any sequence of actions for any task) could be done in the same way, as the loom — as “punched cards” that he called “**programs**”. In this definition, a program involves both the software and hardware for “getting it done” — for executing a plan. Collections of programs can be stored and accumulated and inserted into a machine as desired, hence serving as a kind of “memory” (e.g., just as a weaver may accumulate “programs” (punched cards) for many fabric patterns). Finally, given a complex problem, Babbage noted that programs could mechanize only subsets of tasks. Hence, intelligent problem-solving ultimately involved learning a collection of programs (i.e., plans) for subtasks, and learning to select specific programs for specific tasks as needed.

2.3 Effective procedures as measures of program complexity

Babbage’s proposals imply a complementary view of the complexity of a problem (i.e., of the challenge of credit assignment) from our previous session, not in terms of the amount of information required but the number of actions to take — that is, the complexity of executing plans or programs. We will refer to this measure as **procedural complexity**, in that the sequence of actions for executing a plan is essentially a step-by-step procedure — and of the minimum number of actions for a plan or program as the minimal **effective**

procedure. The standard way of measuring the effective procedure was introduced in 1936 by the mathematician and co-founder of the field **Alan Turing**, which he illustrated through a mathematical model of a hypothetical computing machine (also called an **automaton**), now called a **Turing machine**.

The Turing machine consists of a memory in the form of an infinite tape of symbols of 0's and 1's that can be inputted into each “cell” of the tape. You should note: *the symbols may look similar to the 0,1 bit strings in Shannon information, but this is just an example; the values of the cells in a Turing machine can contain any symbol.* The Turing machine has a “head” (depicted as the bold square below, on the middle cell), which can do any of three **operations**: (1) read the symbol, (2) modify the symbol, or (3) move left or right.

Programs consist of logical rules for executing sequences of such operations. A program to change the five symbols in the cells above to their opposite (X,0,X,1,X,1,0,0,X,X,X → X,1,X,0,X,0,1,1,X,X,X), for example, would consist of a procedure for the Turing machine to move left or right and modify the five cells by flipping their values from 0 to 1 or 1 to 0. Each move or modification of a cell is said to **transition** the machine into a new **state** of the machine. A state refers to the current values of each cell after a particular move and the current location of the head.



A slice of the “infinite tape” of a Turing machine

The complexity of a program can be measured as the number of states in which the program needs to make moves or modifications (i.e., the length of the “effective procedure”). Hence, the complexity of a problem refers to the **amount of states** that a program needs to transition through. The example above illustrates just a tiny program; Turing and his successors in theories of computation have shown mathematically that as the program size grows, the complexity of moves and modifications **grows combinatorially**.

3. Search processes

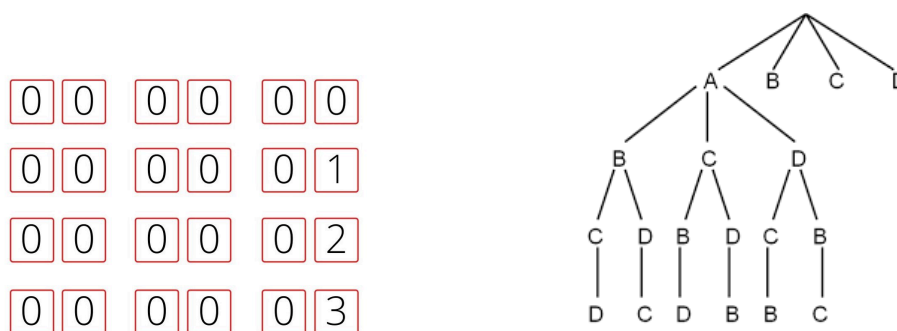
Let us return to our earlier depiction of the process of search as structured into nodes arranged from top to bottom like a hierarchical organization. You can think of the nodes as the possible “**states**” of a problem, and now refer to this picture as a **search tree** (or, equivalently, a *state space* or *search space*). In a search tree, a state refers to a description of a problem adequate to guide an agent’s actions towards a goal akin to the states of a Turing machine. A state should provide enough information (intermediate feedback) for

an agent to trigger some **heuristic** (rule-of-thumb) that guides its selection of the next action. Agents start in an **initial state**. They then generate actions to “transition” the agent to **intermediate states** until (hopefully) eventually reaching a **goal state**. The credit assignment challenge is how to make such transitions efficient, which we look at next as based on heuristics (3.1), including a special heuristic called **satisficing** (3.2).

3.1 Search as a heuristic process

We will call any process of transitioning from an initial state to a goal state in a search tree as a **search process**. Agents search by generating, evaluating, and selecting among plans — i.e., sequences of actions that transition them from an initial state toward a goal state — using heuristics to guide which plans (or parts of plans) to generate-and-test and which to ignore.

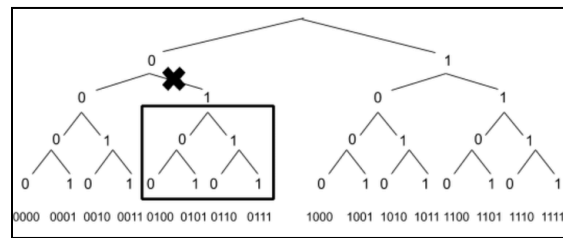
Why call this process “search”? The term is meant to evoke how, given limited processing capacity of agents and the combinatorial complexity of a problem’s search tree (e.g., complexity of the “effective procedure”), agents cannot evaluate every possible action or path. Hence, we use this term **by contrast to brute force**: *rather than describe and test out every state, search involves economizing by using heuristics to make educated guesses about which actions to try, evaluating only parts of the search tree*.



Generating-and-testing combinations for opening a safe. Different paths to test safe combinations (e.g., “path A” could be all possible solutions beginning with “0”).

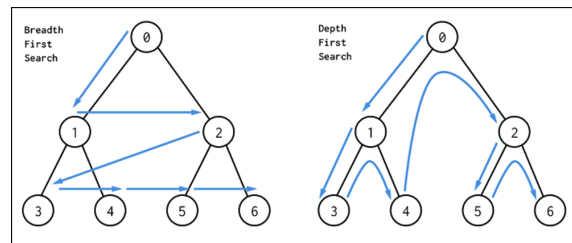
Consider, for example, searching for a sequence of six numbers needed to open the safe depicted above. If an agent tries to identify the solution through brute force, this would involve trying out as many as 1 million combinations (if the possible values are 0-9), starting with (0,0,0,0,0,0) and going as far as (9,9,9,9,9,9). If the combination for the safe is set randomly, then this complexity cannot be avoided. In most problems, however, there are ways of learning or acquiring heuristics that dramatically reduce the size of the tree that has to be searched. This effect of search is called “**pruning**” the search tree, in that using **heuristics** — essentially, rules that guide or stipulate what actions to take or

consider at a particular node — often has the effect of eliminating large sequences of actions or paths (also often called “**branches**”).



“Pruning” a “branch” of a search tree.

Search processes may be informed by a variety of heuristics. As depicted below, they may be generic strategies in an AI system, such as “breadth-first” (search many possible paths, only generating-and-testing a few actions for each path), or “depth-first” (search only one or a few paths, but generate-and-test many actions for each path). Heuristics may also be rules learned by organizations based on specific experiences. For example, a management team might learn rules of testing a product in at least two markets before investing in scaling it.



Breadth- versus depth-first

3.2 Search as a satisficing process

Given that an agent cannot evaluate every state of a search tree and must use heuristics, it *cannot be guaranteed of finding the best plan for solving a problem*. Based on this observation, Simon famously proposed that intelligent agents “**satisfice**”, meaning that they search only for **satisfactory plans** for solving problems, rather than for optimal plans or by maximizing rewards. Satisficing is itself a heuristic, corresponding to a rule of “generate plans until a satisfactory solution is identified, then stop generating plans”.

Simon proposed his theory of satisficing based, firstly, on economic assumptions about information processing costs that hold under either Shannon or Simon information. Plans for solving problems become “good enough” (satisfactory) for an agent when economizing on the time and effort involved in searching for better plans outweigh the benefits of doing so. For this to occur, plans must be close enough to a goal state to cross a satisfactory threshold, or what Simon called an agent’s “**aspiration level**”. For

example, a management team might have a satisficing heuristic of withdrawing from efforts to innovate a new product if profits fall under its aspiration level for six months.

Second, satisficing also follows from Simon's **physical symbol system hypothesis** that we looked at in the previous session. Given information about states of a search tree are physical symbols, they only designate objects or concepts in the external world, they do not constitute it. This physical world, he emphasized, will always have vastly more information than any state descriptions. Problems are therefore always solved based on **limited information** relative to the real world, even given seemingly unlimited "channel capacity". Agents simplify descriptions of states using what he called **models** of the world. As mere simplifications, such models satisfy rather than optimize.

Finally, Simon emphasized that satisficing arises from how complex problems have **goals** that are inherently *organizational*. By organizational, he meant that goals are multiple, often contradictory, and often qualitative in nature and hence ambiguous in terms of the supposed "optimality" of a particular solution. For example, fulfilling the mission of an organization to "contribute to society" could have multiple interpretations, which may vary by department and be mapped to hundreds or even thousands of metrics. Hence, plans used in search are really better thought of as like **organizational programs**, where the complexity is not just that implied by a Turing machine but extends to how processes for learning and selecting these programs are bound up in complex organizational goals that map to problems with no clearly "optimal" solution.

3.3 Paradigmatically-different approaches to search

This section has defined search in general terms as a heuristic process of generating and evaluating satisfactory plans to solve problems. You can understand search as a general strategy for learning to make and select plans (programs) for solving problems in an AI system or organization more efficiently, given the view of complexity implied by the Turing machine. At the same time, this section has also illustrated how the complexity of problems that search is meant to address goes beyond what is implied by the Turing machine or our earlier example of the pin factory. How we conceive of search depends on what we assume when we talk about the states in a search tree and what we assume about how agents can evaluate them as a basis for making and selecting plans.

In the next two sections, we will see that such assumptions can build on either paradigm that we have looked at so far — reinforcement learning and Shannon information, or learning in a society and Simon information. What we will call **von Neumann search** builds on the former, conceiving of search as akin to playing a well-defined game. What we will call **Simon search** builds on the latter, conceiving of search as understanding and structuring an ill-defined problem.

4. Von Neumann search: search as game-playing

A first approach to search derives from, among others, the seminal work of the mathematician John von Neumann in the 1930s and 1940s on **game theory**. You can think of a game as simply a problem where there are two or more agents whose actions are interdependent, yet who have conflicting goals. Put another way, the problem-solving environment for an agent is not just external to it, but can be outright **hostile**. Hence, agents' search for a sequence of actions must consider the strategies of "opponent" agents. This approach to search has focused on strategies for playing various well-defined games, from tic-tac-toe to chess, Go, backgammon and many others.

4.1 Why games?

Why games? Unlike a simple division of labor problem, such as how to plan the sequences of three or so tasks in a pin factory, the adversarial nature of even seemingly simple games results in a *combinatorially-explosive set of states* and, hence, problems that cannot be solved by brute force alone. On the other hand, games, at least of the sort studied in von Neumann search, have *well-defined rules*, which enables precisely studying search processes using mathematical approaches that appealed to von Neumann and many other mathematically-oriented researchers.

In the AI field specifically, the assumption is that AI systems that learn how to play relatively complex games such as chess or Go can serve as stepping stones for AI systems that learn any problem, such as protein folding or science in general. In management research, a further appeal of games has been for **economists**, who view the adversarial nature of a game as akin to how a manager has to make decisions under uncertainty and complexity in a competitive environment.

A disadvantage, as we shall see, is that this game-based approach to search mostly abstracts away from the intuitions about the division of labor and programs that we discussed in Section 2 above. Indeed, AI textbooks do not typically connect the idea of search to examples such as the pin factory; discussions mostly focus on *algorithms*, rather than their instantiation in programs; and typically this game-based approach makes what is called a **unitary actor assumption**, in that it does not matter much whether one assumes that nodes are assigned to different agents — the entire search process is controlled by a single centralized agent.

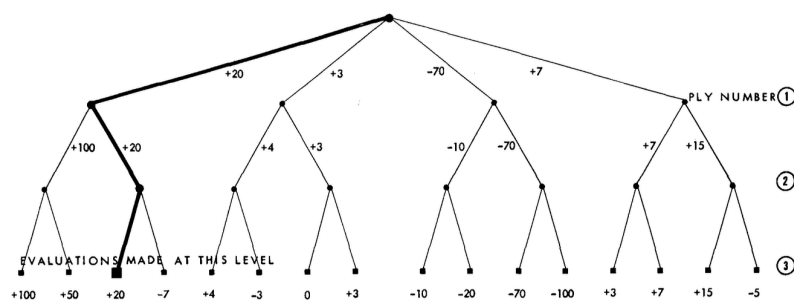
4.2 Heuristics defined in terms of economic utility functions

In games such as chess or backgammon where all the rules and, hence, goals are defined, *there is no inherent need for physical symbols*. All the information about the game can, in

principle at least, be defined in terms of meaningless *bits*, consistent with Shannon information. Likewise, all goals and intermediate feedback can be described without any physical symbols, such that all states can be evaluated with respect to some numerical reward value. In economics, and in game theoretic versions of search in the AI field, this reward is called the **utility** for an agent. Hence, the search tree is really a purely syntactic **game tree**, in which nodes are mapped to numerical values of the expected rewards for pursuing a path through what is called a **utility function**.

To see this utility function in action, let us consider the figure of a search tree again, but now instead of mice and T-mazes, the nodes will be moves in the game of checkers, as in Arthur Samuel's famous checkers program from 1959, mentioned in our session on credit assignment, depicted below. Each path or "branch" of the tree refers to a particular sequence of states of the checkers game from the perspective of one player. We will call one player "Max", since she or he will try to maximize the utility values shown in the figure, while the opponent "Min" wants to minimize these utility values.

Max "explores" the game tree by starting from the top and considering the expected value (utility) of the first move. Since 20 is higher than 7, Max selects that direction and, if using a depth-first heuristic, then explores that path. Since it is combinatorially explosive, Max cannot explore the whole game tree, it uses its **utility function** to calculate the rest of the nodes for the branches that Max considers, then sends this information back up to decide on a path, a process called **backward propagation**, or just **backprop**. Max then selects the move with the highest value ("utility").



In the above game tree, how are the scores for the nodes assigned? In von Neumann search, this is estimated based on what is called an **evaluation function**, which you can think of as some collection of various amounts of intermediate feedback, such as counting up the pieces remaining, the kings remaining, or any other information that is found to be statistically correlated with the probability of winning. Such a function is heuristic, in that it is calculated not by evaluating all the states of the game tree (which would be combinatorially explosive), but by giving an estimate of the value of each state based on various heuristic measures of intermediate feedback. By collapsing these

heuristic measures into a single numerical value for the “utility” of each state, alternative actions can be ranked to guide the search process (i.e., path selection).

4.3 Satisficing by extending the reinforcement paradigm of maximizing rewards

In von Neumann search, since the state of a game can be estimated using the evaluation function as a single “reward”. Hence, under von Neumann search, where all the rules of a game are well-defined such that it is consistent with Shannon information, the learning mechanism is naturally from the reinforcement paradigm (and which may extend to other machine learning techniques that we will cover later in the course). Samuel’s checkers program, for example, famously used reinforcement learning to improve the evaluation function itself. Hence, as the checkers program played many games, it learned heuristics about which positions on the board are good.

The AI researcher Demis Hassibis, head of Google’s DeepMind, and creator of its famous AlphaGo program for playing the game of Go, and AlphaFold for predicting protein structures, has adopted a lot of this basic set-up, combining reinforcement learning with a type of von Neumann search. Based on seeing such progress, Hassibis’s point of view is that combining various search and reinforcement or related learning strategies will be sufficient for intelligence. Hence, what becomes important are many interacting heuristic functions, tied together through the ideas of search and reinforcement or similar learning strategies, all broadly within the von Neumann approach to search. You may contrast this with Richard Sutton’s preference to focus on his *reward hypothesis*, which focuses on collapsing all such search and other functions to a single scalar for reward.

5. Simon search: search as means-ends analysis

A second approach pioneered by Simon, along with his colleague at Carnegie-Mellon Allen Newell, developed by way of contrast with von Neumann-style search and its emphasis on well-defined game trees. Simon’s focus was instead to start with realistic models of how managers make decisions under uncertainty. The distinction that Simon made was that managers have an **open-ended world** in front of them, rather than a complete environment determined by well-defined game rules. For Simon, this open-endedness meant search was not just about coming up with a utility function, and corresponding evaluation function, but rather of the ability to *manipulate physical symbols* to transform some simplified model into a model of some meaningful goal state.

5.1 Heuristics as general methods for manipulating physical symbols

In Simon search, heuristics are used to inform actions for manipulating physical symbols in programs. Specifically a program would have: (i) A set of *meaningful physical symbols*

organized based on lists, and which can be transformed into descriptions of intermediate states; (ii) A set of *operators*, which refers to the actions for transforming symbols that an agent can take; (iii) A set of *rules* regarding how operators transform symbols, which determines how actions transition the agent to another state. When an operator is applied, then the corresponding rule is invoked, and the symbols are transformed.

When an agent takes an action (invokes an operator) that does not immediately lead to the goal state, how does the agent figure out what to do next? Newell and Simon's solution was that, if two states are not equal (i.e., current state and goal state), then this difference can be thought of as a measure of intermediate feedback. The agent can then quantify the feedback about the **difference** between the current state and the goal state. The particular difference is then linked to a **heuristic**, and the heuristic invokes an action likely to get the agent closer to the goal (to a new and better intermediate state).

Newell & Simon called their system the “General Problem Solver”, or GPS, as a model of intelligent search (e.g., search with learning). In GPS, information is generated in the form of **differences** between the **current state** and the **goal state** or any identified **subgoal** states. To illustrate the idea, Simon gave the example of solving algebraic expressions. For example, consider how to solve for X in the algebraic expression:

$$5X+3=2X+7$$

One could do naïve “generate and test” (pure search) by simply generating different possible values for X and seeing if both sides of the equation are equal. Yet, assuming we do not make a lucky guess, a generate and test strategy would not provide much information (feedback), other than that the previous guess was incorrect. Instead, intelligent problem-solvers would first rearrange the expression to read:

$$5X+3-2X-7=0$$

In this case, one can use a generate and test strategy more effectively as a source of information or feedback. That is, since the X variables are all on one side, we can see how close each guess gets to 0 and gradually improve our guesses, reducing the amount of possible values to search. Still more effective, however, is to see that the above algebraic expression is a general example of $X = Y$, and thus to rearrange with X all in one coefficient:

$$3X=4$$

and then to:

$$X = 4/3$$

Once the problem is described in this state, one can find the solution of $X = 4/3$ *with no search at all*. The key idea is that **the goal state is already represented in the initial state**. The operations for solving the algebraic expression were essentially symbol manipulation by reasoning using the rules of algebra — that is transforming the initial set of symbols into the goal state. In information processing terms, all the information is given by the initial state and the rules about the domain that we assume.

As illustrated by the simple but powerful transformations in this algebra example, Newell & Simon's idea in GPS was to boil down the vast universe of possible heuristics to their essentials — to **heuristics for generating heuristics**. They proposed, for example, that there were fundamental heuristics at play, which they referred to as fundamental “methods”, or “schemes” for setting up subgoals, and which describe the essence of heuristic search, and include: (i) “*Pattern-matching*”: First, given a goal of transforming object A into B, then the objects are matched and “compared element by element”. If there is a difference among one or more elements, then a subgoal is generated of reducing this difference; (ii) “*Generate-and-test*”: Second, select an operator, and try it out based on the given rules, then apply the same pattern matching process and see if A is now closer to B. Hence, in search, learning works by developing fundamental heuristics that, given any goal state and initial state, could generally construct and “discover” new heuristics through the search process itself.

5.3 Learning as the use of search operations to acquire new heuristics like an expert

Simon realized that a central challenge was how an AI system could have enough diverse experiences to learn heuristics for solving any problem. This challenge of learning to acquire heuristics became regarded among scholars of search as the central “bottleneck” to intelligence. One solution is simply to hand-code the heuristics by interviewing human experts and having them articulate them, a line of research called **expert systems**.

Simon's idea, however, was not to accumulate many heuristics, but rather to closely study how experts across many fields solved problems (e.g., chess grandmasters, scientists, athletes) and *generalize these findings into a model of search by experts* — essentially, to identify a small number of highly general heuristics such that an agent could learn to acquire heuristics on its own. Subsequent research by his students found, for example, that experts in various fields used heuristics related to “deliberate practice” to learn other heuristics, or that entrepreneurs used heuristics to construct business ideas in a piecemeal fashion (a process called “effectuation”) rather than plan things out ahead of time.